

Course

« *Deep Learning* »

Sid-Ahmed Berrani

ENSIA

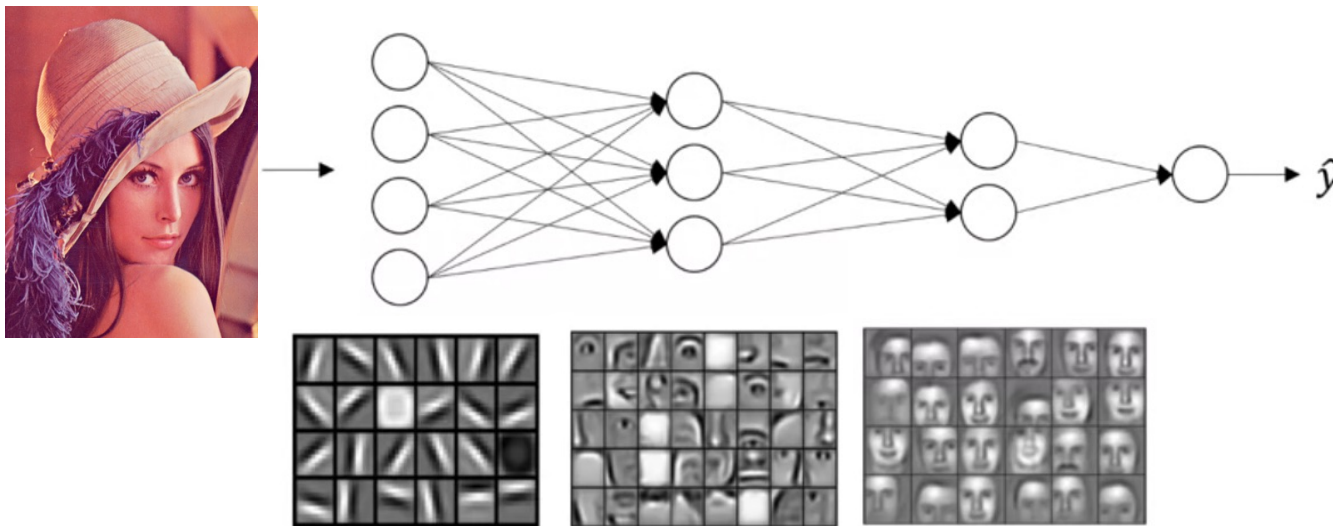
2024 – 2025

3. Deep Neural Network

- Now, you know all what is needed to build a DNN :
 - Neurons.
 - Activation functions.
 - Loss and cost functions.
 - Forward propagation.
 - Backward propagation.
 - How to prepare and how to split the dataset (train/dev/test).
 - Why and how to normalize the data.

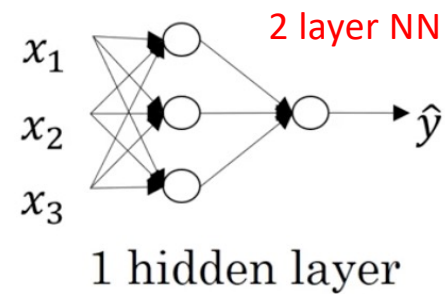
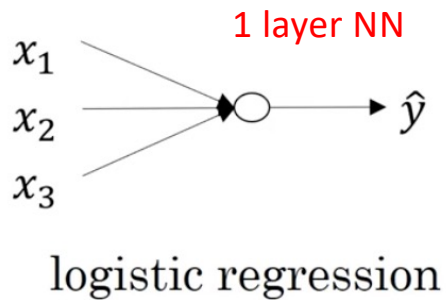
3. Deep Neural Network

- Why deep models?
 - The earlier layers of the neural network: Detection of simple functions, like edges.
 - These are then composed together in the later layers.
 - NN can thus learn more and more complex functions.

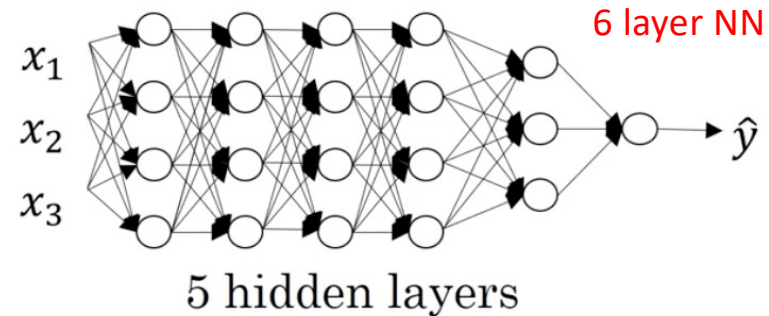
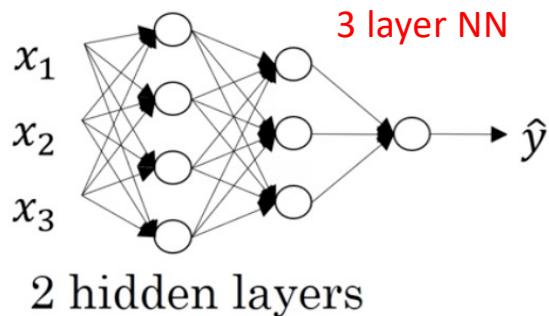


3. Deep Neural Network

- Now, you know all what is needed to build a DNN



The number of layers
is a hyper parameter



3. Deep Neural Network

- **Notations**

Number of layers : $L = 4$.

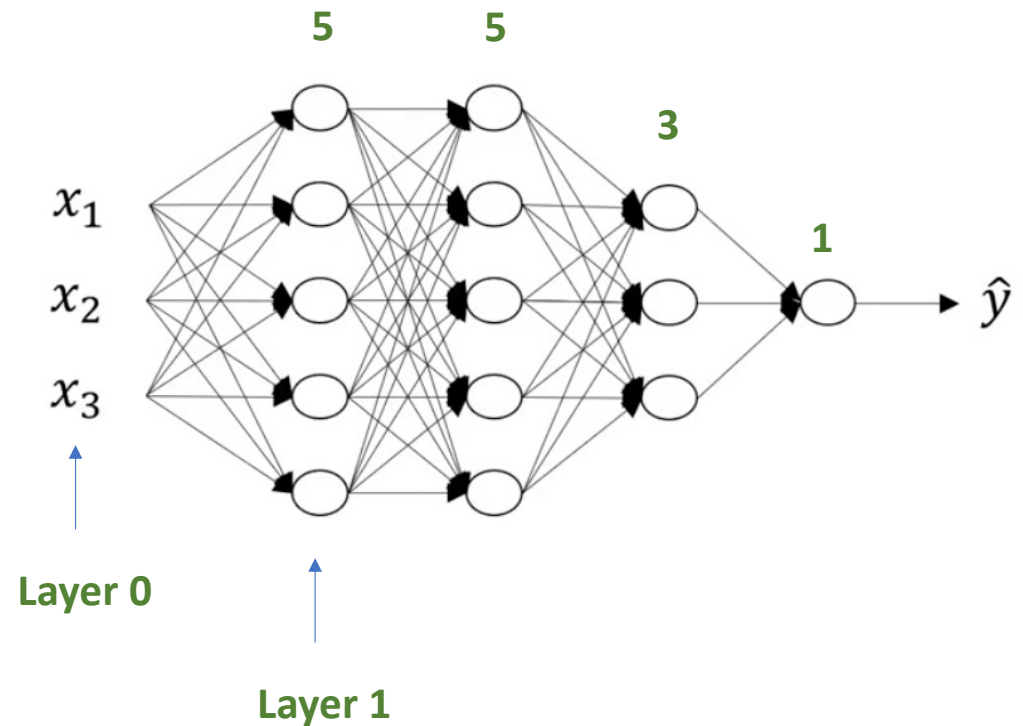
Number of neurons in layer l : $n^{[l]}$

$n^{[0]} = n_x = 3$ $n^{[1]} = 5$...

Activations in layer l : $a^{[l]}$.

$a^{[l]} = g^{[l]}(z^{[l]})$ $x = a^{[0]}$

Weights for l : $W^{[l]}, b^{[l]}$.



3. Deep Neural Network

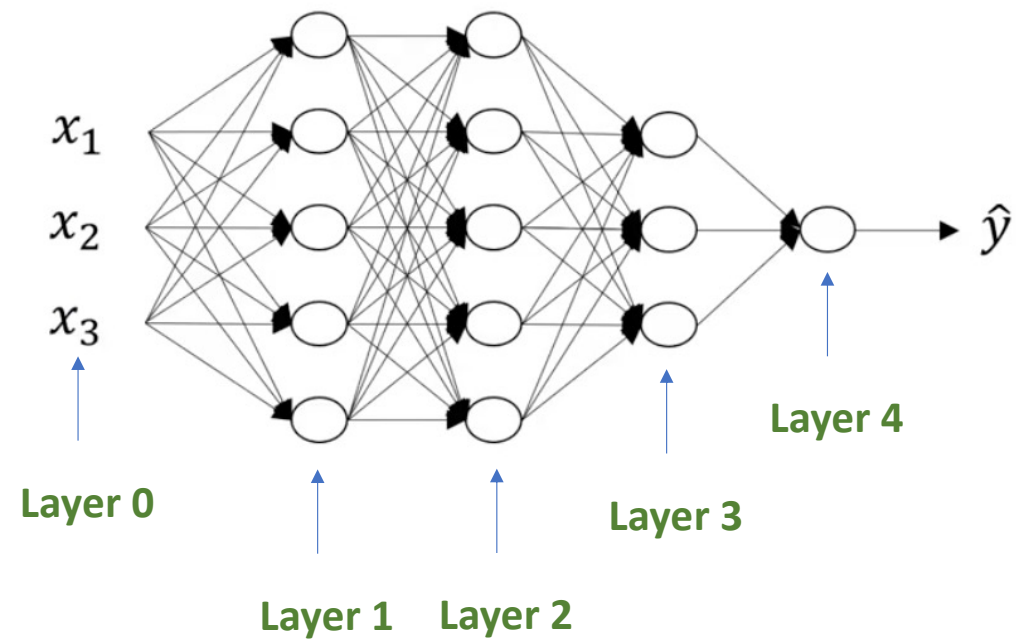
- **Forward propagation**

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

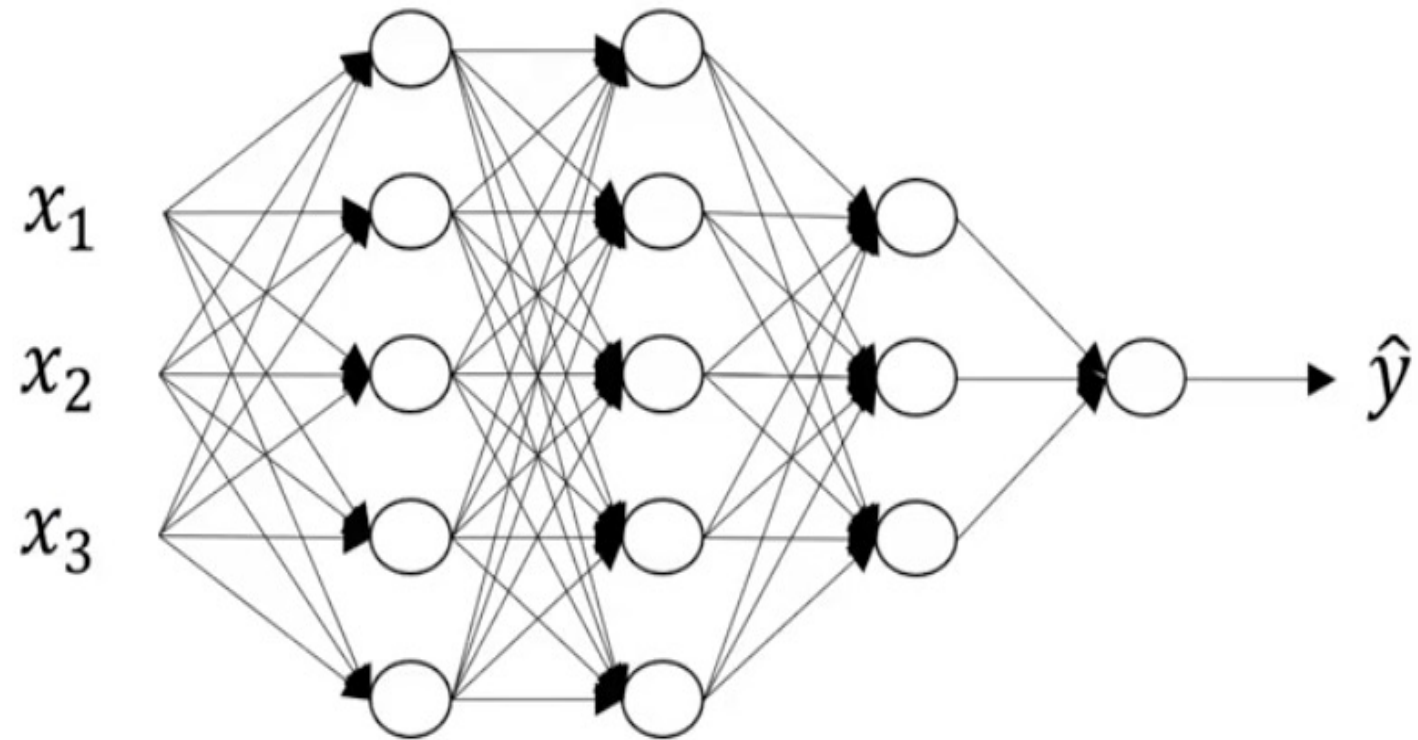
$$a^{[0]} = x$$

$$a^{[4]} = \hat{y}$$



3. Deep Neural Network

- Forward and backward



3. Deep Neural Network

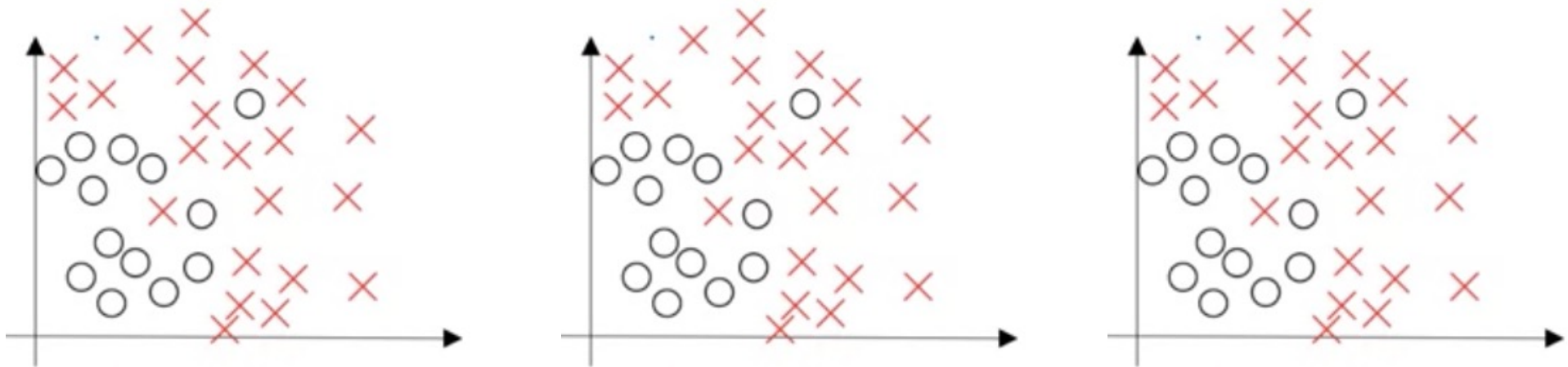
Practical aspects

- Focus on optimization aspects:
 - Understanding bias and variance.
 - Optimization of the learning process.
 - The use of mini-batch gradient descent.
 - Techniques to avoid overfitting.
 - ...

3. Deep Neural Network

Bias & Variance

- Easy to understand and to learn by very difficult to master in practice.



3. Deep Neural Network

Bias & Variance

- **Bias** — High bias means the model is not “fitting” well on the training set => Obtained **performance** after **training** is poor.
- **Variance** — High variance means that the model is not able to make accurate predictions on the validation/test set. The validation error will be large => The **generalization** capacity of the model is **poor**.

3. Deep Neural Network

Bias & Variance

- Two key numbers to consider are:
 - The train set error.
 - The dev set error.

This analysis assumes a human level error = 0%

- Let's consider the following situations:

- | | | | | |
|---------------------|------|-----|-----|----|
| • Train set error : | 0.5% | 18% | 18% | 2% |
| • Dev set error : | 10% | 19% | 35% | 3% |

High
variance

High
biais

High biai &
High variance

Low biai &
low variance

3. Deep Neural Network

Bias & Variance

- Two key numbers to consider are:
 - The train set error.
 - The dev set error.

But if the human level
error = 16 %

- Let's consider the following situations:

- | | | | | |
|---------------------|-----|-----|-----|-----|
| • Train set error : | 8% | 18% | 30% | 30% |
| • Dev set error : | 25% | 19% | 32% | 45% |

High
variance

Low bias &
low variance

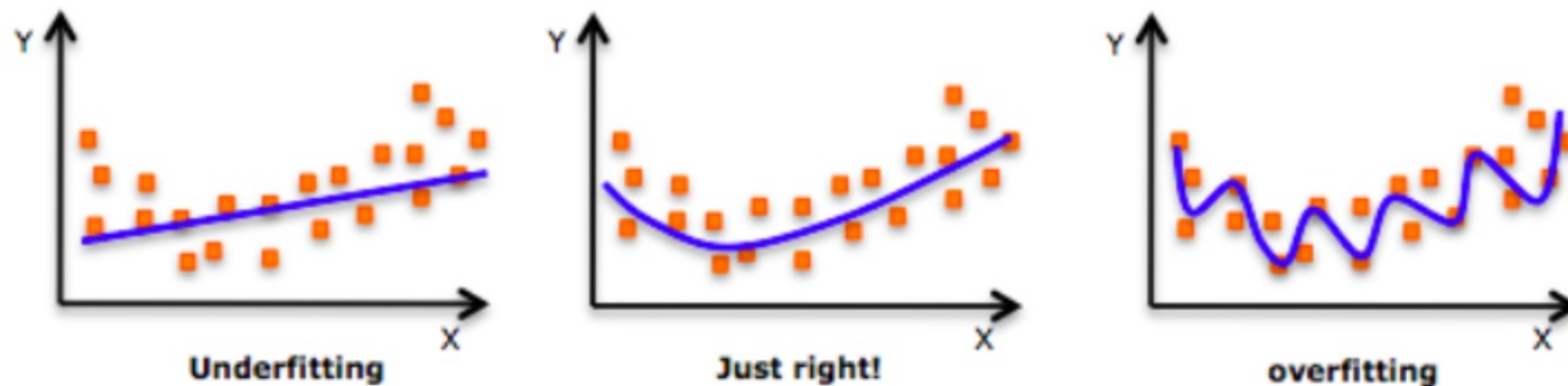
High
bias

High bias &
High variance

3. Deep Neural Network

Bias & Variance

- High bias = underfitting
- High variance = overfitting

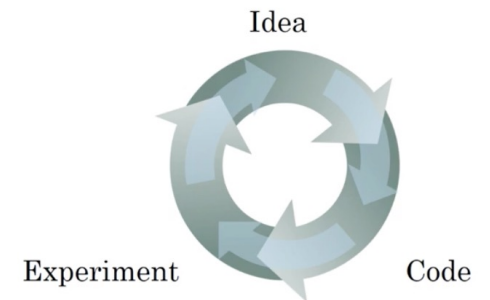


- Diagnosis is based on the analysis of training & dev errors.

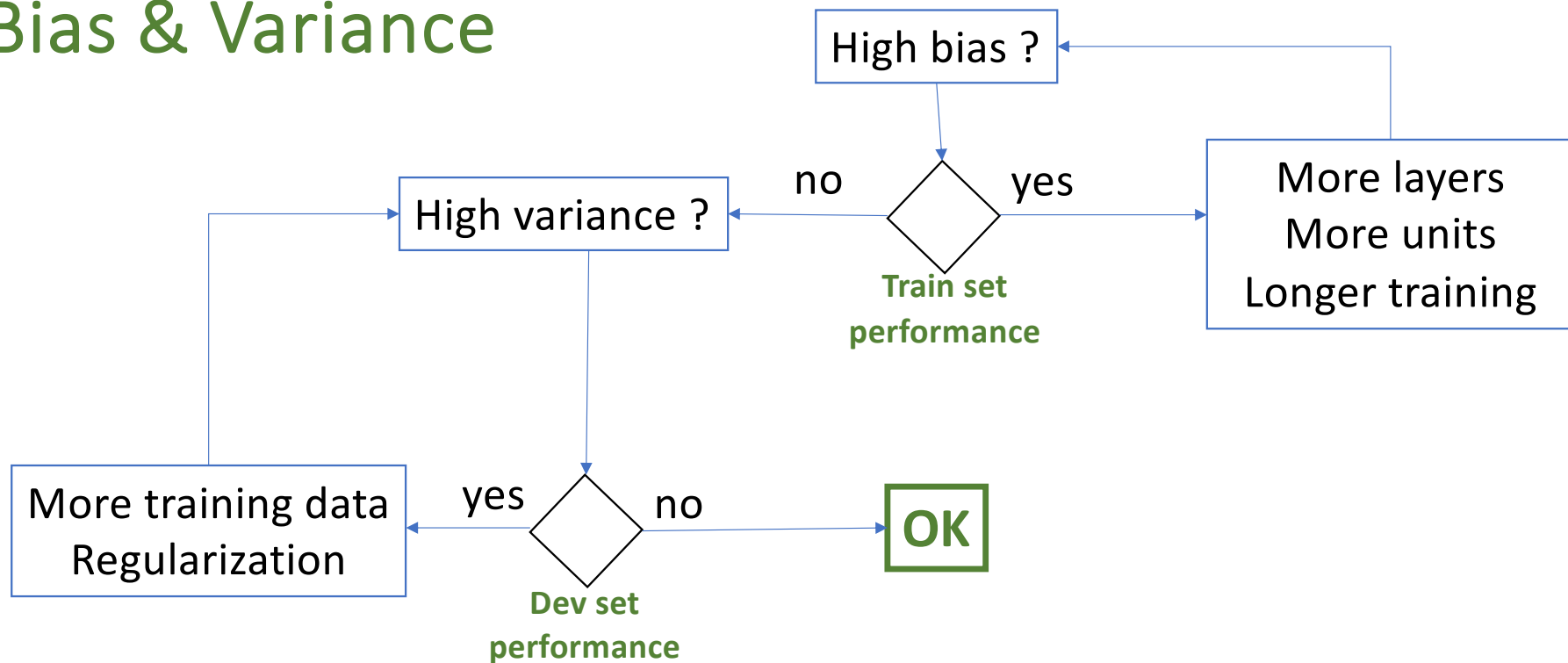
3. Deep Neural Network

Bias & Variance

- The question:
 - What to do in case of high bias, high variance or both?
 - What methodology do we have to follow to deal with the problem?
- Remember, training a DNN model is an iterative process.
- Bias & variance are one of reasons, dev set is important/required.



3. Deep Neural Network Bias & Variance



As long as we can have **more data** and we can **train the network longer** while correctly and effectively using **regularization** => **No need to search for a tradeoff between bias and variance.**

3. Deep Neural Network

Regularization: A Solution to Avoid Overfitting

- Overfitting = high variance.
- To prevent or reduce overfitting => Regularization / adding more training data
- **What is regularization?**
“ In mathematics, statistics, finance^[1], computer science, particularly in machine learning and inverse problems, regularization is the process of adding information in order to solve an ill-posed problem or to prevent overfitting »

[https://en.wikipedia.org/wiki/Regularization_\(mathematics\)](https://en.wikipedia.org/wiki/Regularization_(mathematics))

3. Deep Neural Network

Regularization: A Solution to Avoid Overfitting

Regularization in machine learning [\[edit \]](#)

In [machine learning](#), a key challenge is enabling models to accurately predict outcomes on unseen data, not just on familiar training data. Regularization is crucial for addressing [overfitting](#)—where a model memorizes training data details but can't generalize to new data—and [underfitting](#), where the model is too simple to capture the training data's complexity. This concept mirrors teaching students to apply learned concepts to new problems rather than just recalling memorized answers.^[4] The goal of regularization is to encourage models to learn the broader patterns within the data rather than memorizing it. Techniques like [early stopping](#), L1 and [L2 regularization](#), and [dropout](#) are designed to prevent overfitting and underfitting, thereby enhancing the model's ability to adapt to and perform well with new data, thus improving model generalization.^[4]

[https://en.wikipedia.org/wiki/Regularization_\(mathematics\)](https://en.wikipedia.org/wiki/Regularization_(mathematics))

3. Deep Neural Network

Regularization: A Solution to Avoid Overfitting

- Overfitting = high variance.
- To prevent or reduce overfitting => Regularization / adding more training data
- When doing training, the objective is to minimize the loss.

$$\min_{w,b} J(w, b) \quad (\text{for logistic regression})$$

But in order to avoid overfitting : We have to reduce the **complexity of the model**.

3. Deep Neural Network

Regularization: A Solution to Avoid Overfitting

- The idea is to add a second term in order to penalize model complexity :
Minimise : Loss(Data / model) + Complexity (Model)

The training optimization algorithm is now a function of two terms:

1. The **loss term**, which measures how well the model fits the data,
2. The **regularization term**, which measures model complexity.

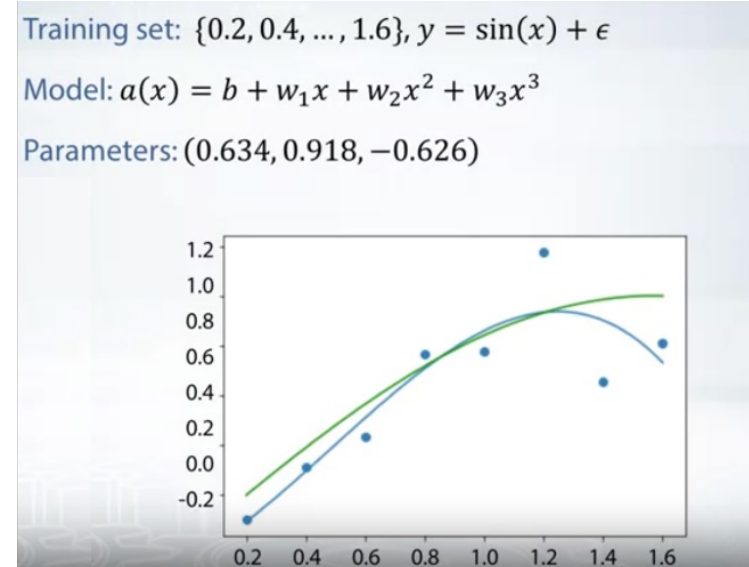
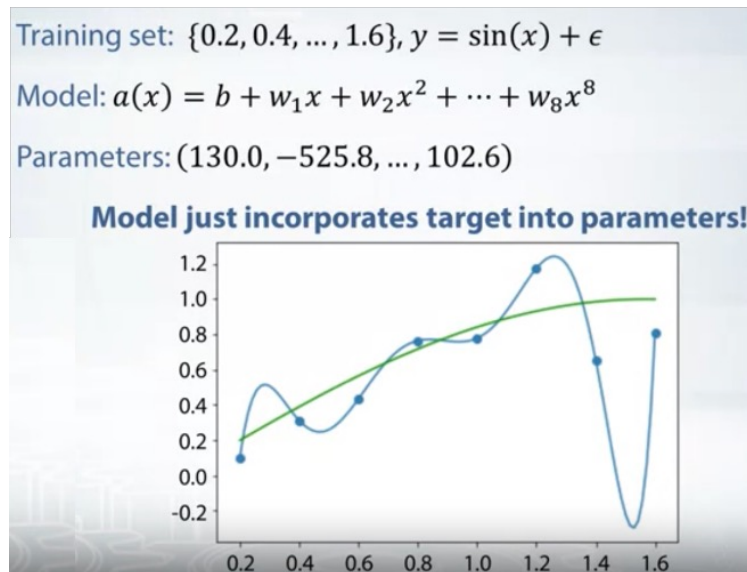
But how to define model complexity?

- Model complexity as a function of the *weights* of the model.
- A feature weight with a high absolute value is more complex than a feature weight with a low absolute value.

3. Deep Neural Network

Regularization: A Solution to Avoid Overfitting

- About complexity



Good model weights: $(0.634, 0.918, -0.626)$

Overfitted model weights: $(130.0, -525.8, \dots, 102.6)$

3. Deep Neural Network

Regularization: A Solution to Overfitting

- Case of logistic regression:

$$\min_{w,b} J(w, b) \quad J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|_2^2 + \cancel{\frac{\lambda}{2m} b^2}$$

$$L_2 \text{ regularization: } \|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2$$

3. Deep Neural Network

Regularization: A Solution to Overfitting

- Case of logistic regression:

$$\min_{w,b} J(w, b)$$

$$w \in \mathbb{R}^{n_x}, \quad b \in \mathbb{R}$$

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|_2^2$$

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{m} \|w\|_1$$

L_2 regularization:

$$\|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2$$

L_1 regularization:

$$\|w\|_1 = \sum_{j=1}^{n_x} |w_j|$$

**λ : regularization parameter
(a hyper-parameter that has
to be tuned)**

3. Deep Neural Network

Regularization: A Solution to Overfitting

- **Why does regularizations reduce the absolute values of weights?**

3. Deep Neural Network

Regularization: A Solution to Overfitting

- λ : regularization parameter

When choosing a λ value, the goal is to strike the right balance between simplicity and training-data fit:

- A λ value too high $\Rightarrow$ model will be simple, but we run the risk of *underfitting* your data
- A λ value too low $\Rightarrow$ model will be more complex, and you run the risk of *overfitting* your data.

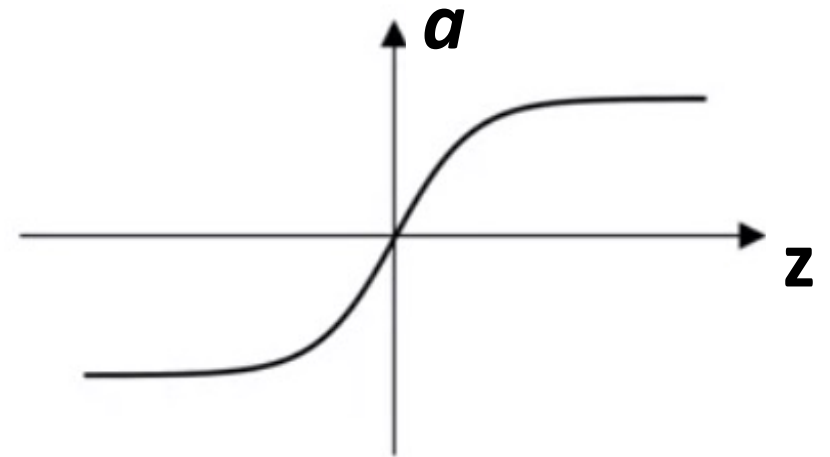
3. Deep Neural Network

Regularization: A Solution to Overfitting



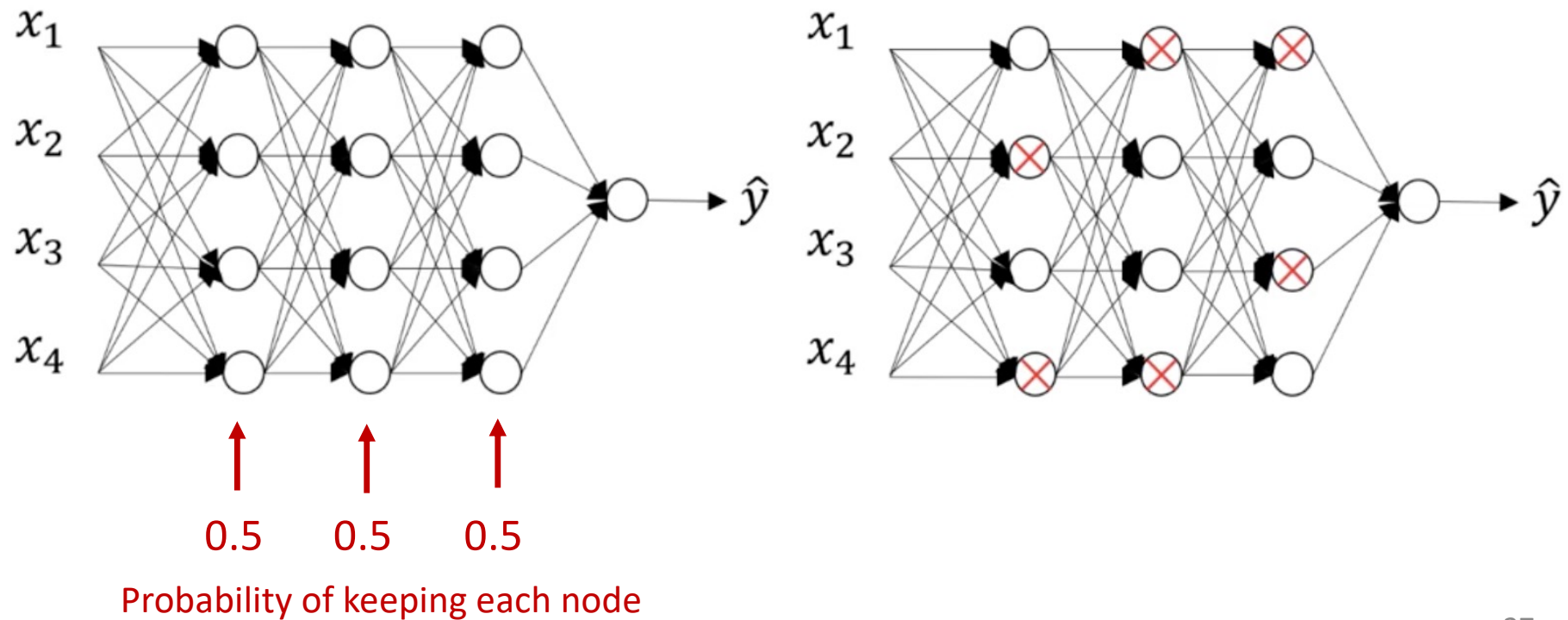
3. Deep Neural Network Regularization: Intuition...

- Activation function: $g(z) = \tanh(z)$.
- Increase $\lambda \Rightarrow$ decreasing w .
- $z = w x + b$
- $a = g(z)$
- Derivative: da / dz



3. Deep Neural Network Regularization: Dropout

- Another very powerful regularization method



3. Deep Neural Network Regularization: Dropout

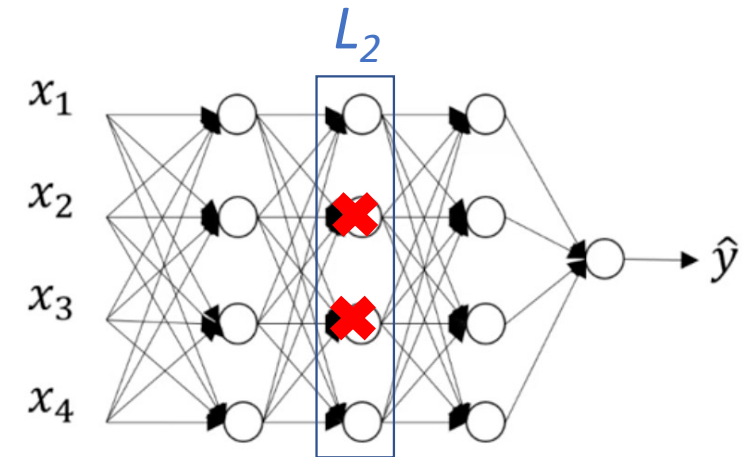
Implementation: Inverted dropout

- Probability to keep a hidden unit : $kp = 0.5$
- Construct a random vector d_2 (using kp)
- The dimension of $d_2 = n^{[2]}$: the number of neurons in L_2
 $\Rightarrow$ For instance : $d^{[2]} = [1 \ 0 \ 0 \ 1]$
- Multiply $d^{[2]}$ by $a^{[2]}$ (element-wise)
- $a^{[2]} = a^{[2]} / 0.5$

$$z^{[3]} = W^{[3]} \cdot \mathbf{a}^{[2]} + b^{[3]}$$



Reduced by 50 % due to dropout



3. Deep Neural Network Regularization: Dropout

- And what to do at test time?

Nothing!

Thanks to “ $a^{[2]} = a^{[2]} / kp$ ” that used during training

=>

The expected value of each activation remains the same.

3. Deep Neural Network

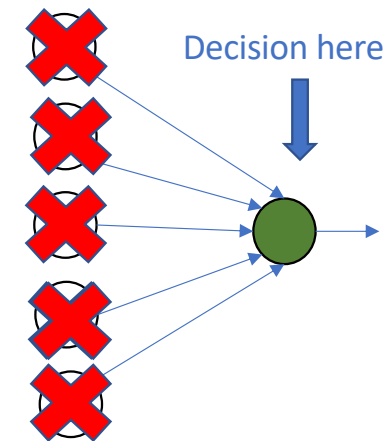
Regularization: Dropout

Why does it work in practice as a *regularizer*?

- One explanation: At each iteration, we work with a smaller network.
- A smaller NN => Less complexity in the model => Low risk of overfitting
- Another explanation:

Dropout insures that none of the input neurons is essential.

The decision does not rely on some specific units.



3. Deep Neural Network

Regularization: Dropout in Practice

- It is possible to have a different dropout parameter for each layer.
- It is technically possible to apply dropout to the input layer, but almost never used in practice.
- The field where dropout is very successful:

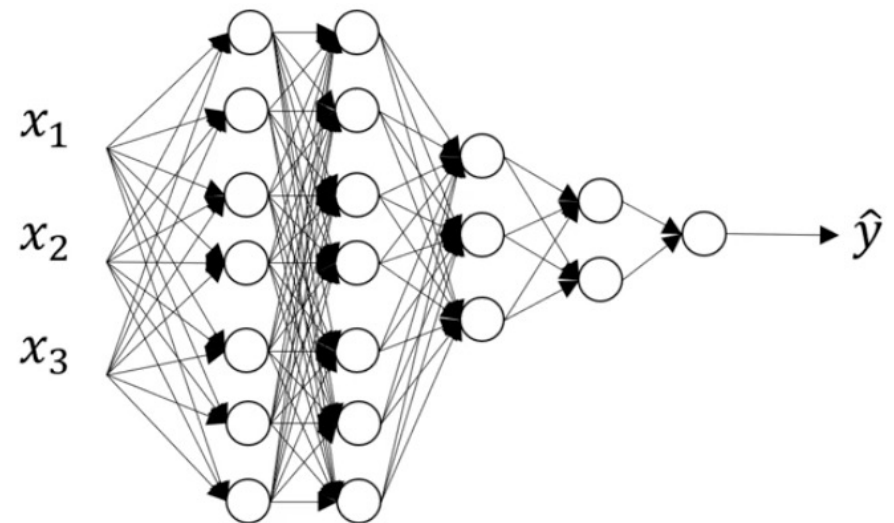
Computer vision

Lack of training data

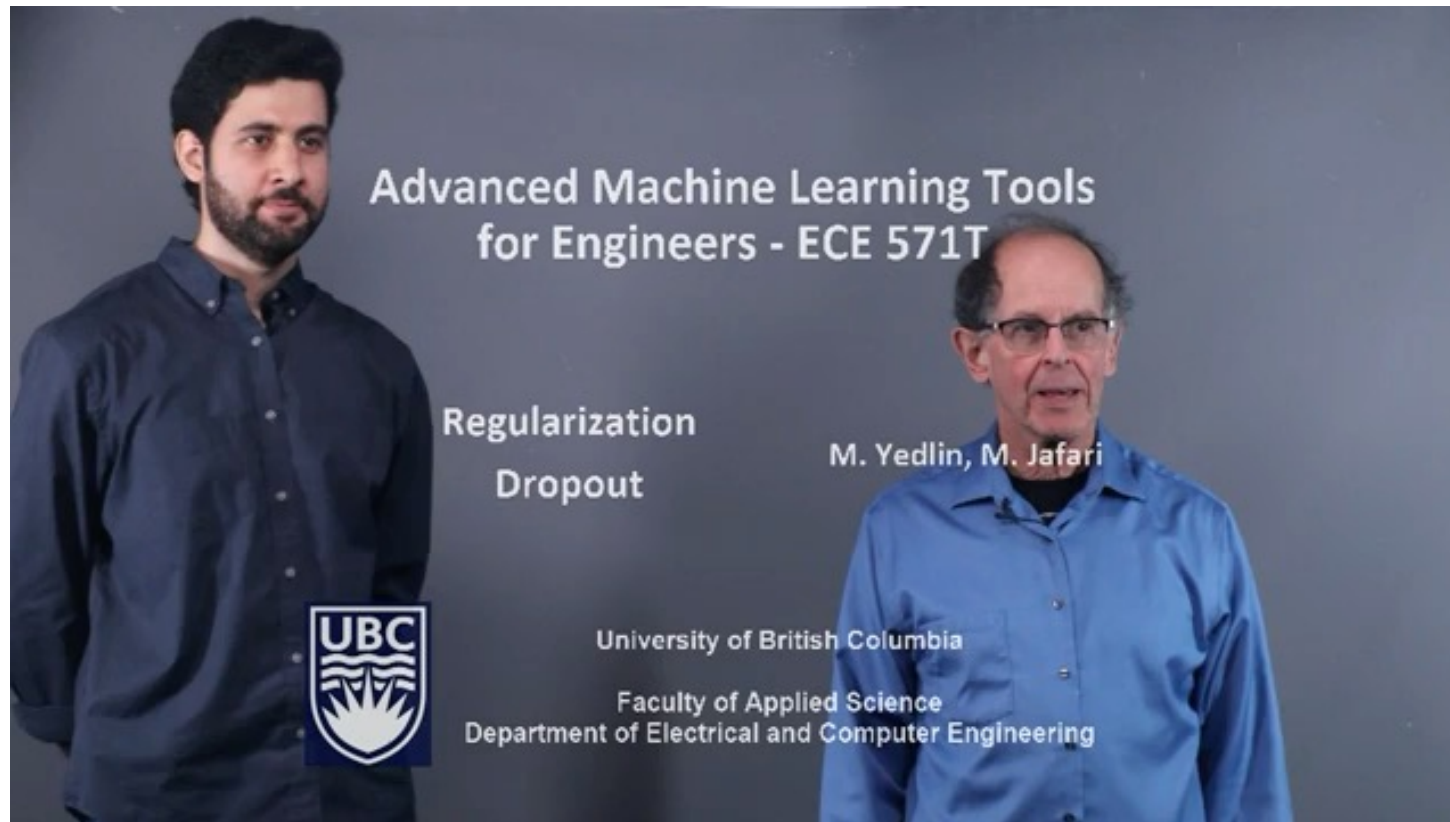
Very high-dimensional data

Complex models.

- In practice, dropout has to be used only when facing an overfitting situation.



Video about regularization - Dropout - 1



Video about regularization - 2

Neural Networks for Machine Learning

Lecture 10e

Dropout: an efficient way to combine neural nets

Geoffrey Hinton
Nitish Srivastava,
Kevin Swersky
Tijmen Tieleman
Abdel-rahman Mohamed

